DATABASE SYSTEMS · COURSE PROJECT 1

基于 MongoDB 的在线问卷系统

数据模型选择、规则引擎与演进能力设计

第一阶段项目完成报告 | 2026 年 10 月

姓名：杨子炫 学号：10245501483

Luli3220/Dase_DataBase_Porject_I

摘要 本项目设计并实现了基于 MongoDB 的在线问卷系统，支持用户注册登录、问卷创建与发布、单选、多选、文本和数字四类题型、条件跳转、匿名保护、重复填写及结果统计。针对问卷结构灵活、历史版本持续演进和填写记录不断增长的特点，我们经过方案比较，采用四集合数据模型，将题目、选项及跳转规则嵌入问卷版本文档，并将答卷独立存储，以兼顾数据组织的合理性与后续扩展需求。在此基础上，系统通过版本冻结与发布指针保留历史问卷，通过声明式规则引擎和后端路径校验保证填写逻辑的正确性，并结合事务、幂等机制及跨版本统计策略维护数据一致性。项目完成了单元测试、MongoDB 集成测试及浏览器端到端测试，最终 113 项测试全部通过。本文进一步总结了数据库方案的设计取舍、AI 辅助开发中的问题与改进，并讨论了第二阶段的扩展方向及现有方案的适用边界。

1 项目目标与需求分析

1.1 功能需求分析

本项目要求开发一个基于 MongoDB 的在线问卷系统，支持用户创建、发布、填写问卷及查看统计结果。根据课程需求，系统主要涉及问卷创建者和填写者两类参与者，同一注册用户可以同时具有两种身份。

对于问卷创建者，系统需要支持问卷创建与管理，包括设置标题、说明、匿名填写和截止时间，以及添加、编辑、发布和关闭问卷。问卷支持单选、多选、文本和数字填空四种题型，并允许为不同题型设置相应的答案约束与条件跳转规则。

对于填写者，系统要求用户登录后通过公开链接访问问卷，根据预设规则依次回答或跳转至指定题目，并在提交时检查答案是否合法。系统需要保存每次有效提交，同时支持多人填写、匿名填写及按设定策略重复填写。

在结果统计方面，创建者需要能够查看整份问卷或单道题目的统计结果，包括选项选择次数、实际回答人数、文本答案及数字平均值。统计结果应根据已保存的答卷自动生成。

1.2 业务对象与概念关系

根据上述需求，可以识别出用户、问卷、题目和答卷四类基本业务对象，其关系如图 1 所示。

一个用户可以创建多份问卷，每份问卷由一位用户创建，并包含多道题目。用户也可以填写不同的问卷，每次提交形成一份独立答卷；一份问卷可以接收多份答卷。答卷与题目之间通过作答关系关联，一份答卷可以包含多道题目的回答，而同一道题目也可能出现在不同用户提交的多份答卷中。

上述概念模型明确了系统的主要业务对象及其联系，为后续数据库结构设计提供了基础。

1.3 数据库设计目标

本项目分为两个阶段：第一阶段完成基础系统，第二阶段根据新增需求进行修改与扩展，且不得丢弃第一阶段成果重新实现。因此，数据库设计不仅需要满足现有功能，还应尽量降低未来需求变化带来的修改成本。

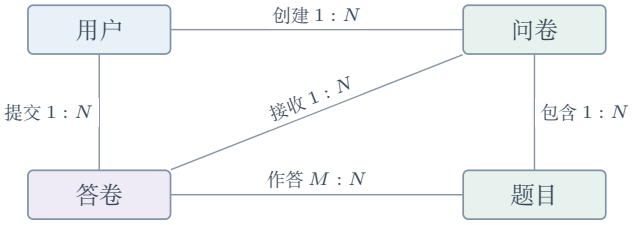


图 1. 问卷系统核心业务对象及其概念关系

基于这一要求，我们主要关注三个方面：**通用性**，使用统一的数据结构表示不同类型的题目与规则，减少针对具体问卷的特殊处理；**隔离性**，根据数据的职责和变化特点划分存储边界，避免局部修改影响无关数据；**可扩展性**，使新增题型、规则及统计功能能够在已有结构上逐步实现，而不必重新组织整个系统。

此外，问卷内容可能在发布后发生修改，而已有答卷仍需要按照填写时的题目和规则进行解释。因此，系统还需要保留历史内容与提交记录，避免后续修改改变旧数据的含义。

在上述需求与设计目标的基础上，第三章将进一步比较不同的 MongoDB 数据组织方案，并确定最终的集合划分、文档结构及关联方式。

2 系统设计

系统采用前后端分离架构。前端使用 HTML、CSS 和 JavaScript 实现问卷编辑、填写与结果展示；后端基于 FastAPI 提供接口，通过 PyMongo 与 MongoDB 交互。后端按照功能划分为用户认证、问卷管理、答卷处理和结果统计等业务模块，题目校验与条件跳转则由独立的规则引擎负责。

系统整体架构如图 2 所示。各模块通过明确的接口协作，将界面交互、业务处理和数据存储相互分离。这种设计能够减少模块之间的依赖，使后续功能调整主要集中在相关模块中，而不必重新设计整体架构。

在数据库层面，不同业务操作对数据的使用方式存在差异。创建和填写问卷时，需要获取完整的题目及规则配置；提交问卷时，需要持续保存用户的填写记录；查看统计结果时，则需要对答卷进行筛选与聚合。这些数据在访问方式、增长规模和更新频率上各不相同，因此需要结合实际业务需求，合理确定数据的组织与关联方式。

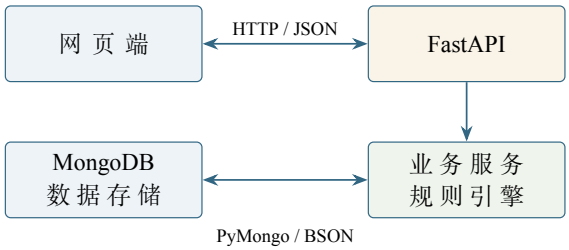


图 2. 系统整体架构与模块关系

3 MongoDB 设计

3.1 候选方案及取舍

数据库结构需要根据实际的数据使用方式进行设计。本项目主要考虑三个问题：如何高效读取完整问卷、如何保留问卷修改产生的历史版本，以及如何管理持续增长的填写记录。围绕这些需求，我们比较了三种 MongoDB 数据组织方案，具体见表 1。

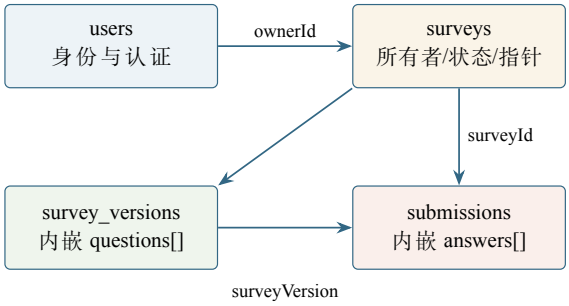


图 3. 核心集合及主要引用关系（逻辑关联）

数据库结构需要根据实际的数据使用方式进行设计。本项目主要考虑三个问题：如何高效读取完整问卷、如何保留问卷修改产生的历史版本，以及如何管理持续增长的填写记录。围绕这些需求，我们比较了三种 MongoDB 数据组织方案，具体见表 1。

为说明不同方案的区别，假设一份问卷最初包含 10 道题目，发布后收到 500 份答卷。随后，创建者希望增加两道题目并调整跳转规则，形成新的问卷版本，同时保留原有答卷。

方案 A（三集合）：将问卷基本信息及各版本的题目配置集中在同一个 `surveys` 文档中，答卷独立存储。这种方式结构简单，获取完整问卷较为直接。但随着问卷不断修改，新旧版本的配置会持续累积在同一文档中，使文档规模和版本管理的复杂度逐渐增加。

方案 B（四集合）：将问卷基本信息与内容版本分开存储。`surveys` 只需记录当前发布版本和草稿版本的指针，每个版本则作为独立文档保存在 `survey_versions` 中。例如，修改后的 V2 可以独立保存，而原有 500 份答卷仍然关联 V1，不会因新版本发布而改变其含义。这种方式需要额外维护版本关联，但能够避免历史版本不断增加导致问卷主文档膨胀。

方案 C（题目独立拆分）：进一步将题目、选项等数据分别存储，通过 ID 建立关联。这种方式便于独立修改某道题目，适合题目数量较多或需要多人协同编辑的场景。但本项目通常需要读取整份问卷的题目与规则，过细的拆分会增加数据查询和组装的工作量；保留历史版本时，也需要额外维护题目、选项与版本之间的关系。

表 2. 四类集合与关键字段

集合	关键字段及职责
users	username（唯一）、passwordHash、createdAt、sessionVersion
surveys	ownerId、publicId（唯一）、status、publishedVersion、draftVersion、submissionPolicy、deadlineAt
survey_versions	surveyId、version、state、revision、settings、questions[]
submissions	surveyId、surveyVersion、respondentId、answers[]、requestId、submittedAt

综合比较，本文最终**采用方案 B，即四集合模型**。该方案在版本内部保留完整的题目配置，同时将问卷基本信息、内容版本和填写记录分别管理。相比方案 A，它更适合长期保留多个历史版本；相比方案 C，又避免了不必要的细粒度拆分。因此，方案 B 能够在完整问卷读取、历史数据保留和后续扩展之间取得较好的平衡。

3.2 四集合职责与文档边界

系统最终采用 `users`、`surveys`、`survey_versions` 和 `submissions` 四个核心集合，其关联关系如图 3 所示，主要字段见表 2。

其中，`users` 统一管理用户身份与认证信息；`surveys` 保存问卷标题、创建者、发布状态及版本指针；`survey_versions` 保存各版本的题目内容与规则；`submissions` 则记录每次填写及其对应的问卷版本。

在具体的数据组织上，我们结合 MongoDB 的文档特点，采用**嵌入（Embedding）与引用（Referencing）相结合**的方式，其具体结构如图 4 所示。

嵌入：对于经常一起读取、需要共同管理且数量相对有限的数据，将其保存在同一文档中。例如，填写者打开问卷时，需要获取题目、选项、校验条件和跳转规则，因此将这些配置嵌入 `survey_versions` 的 `questions[]` 中。这样在确定版本后，只需读取对应版本文档即可获得完整配置；发布时也可以将该版本作为整体冻结，避免后续修改影响历史内容。

同样，一次提交中的多个答案共同构成一份答卷，因此将它们嵌入 `submissions` 的 `answers[]` 中。系统可以完整保存一次填写记录，而不必将每道题目的答案单独存储并重新关联。

引用：对于需要独立管理或数量可能持续增长的数据，通过 ID 建立集合之间的关联。例如，一个用户可以创建多份问卷，一份问卷可以拥有多个版本和大

表 1. 不同 MongoDB 数据组织方案的比较

比较维度	方案 A：三集合	方案 B：四集合（采用）	方案 C：题目独立拆分
集合划分	users、surveys、submissions	users、surveys、survey_versions、submissions	users、surveys、questions、options、submissions
数据组织方式	问卷基本信息及各版本的题目配置集中在同一 surveys 文档中，答卷独立存储。	surveys 保存基本信息与版本指针，每个版本作为独立文档保存完整题目配置，答卷独立存储。	题目、选项分别存储，通过 ID 建立关联；规则随题目管理。
完整问卷读取	问卷与版本配置位于同一文档，通常一次查询即可获取。	先读取问卷的版本指针，再获取对应版本的完整配置。	需要查询并关联题目、选项等多个文档，重新组织完整问卷。
修改与历史保留	多个版本集中管理，历史版本增加会使问卷文档持续增长。	新版本独立保存，已发布版本保持不变，历史答卷仍可按原版本解释。	便于独立修改题目，但需额外维护版本归属及历史快照。
数据独立性与扩展	结构简单，但问卷状态与历史配置共用同一文档，管理较为集中。	问卷信息、内容版本和答卷分别管理，减少不同数据之间的相互影响。	题目和选项可独立管理，适合细粒度编辑及协同操作。
主要代价	版本积累可能导致单文档膨胀，长期版本管理较复杂。	需要额外读取版本文档，并维护版本指针及跨文档一致性。	集合关联较多，完整读取和版本管理的实现成本较高。
综合评价	实现简单，适合版本较少的小型问卷系统。	**兼顾完整问卷读取、历史保留与后续扩展，更符合本项目需求。**	独立性较高，但对当前项目而言拆分过细。

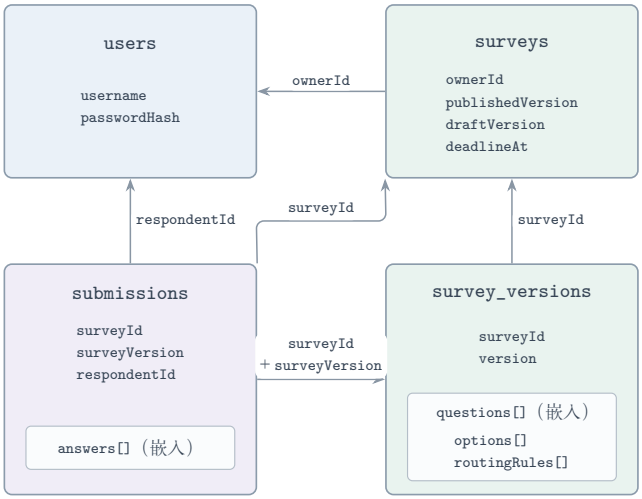


图 4. 问卷系统中嵌入与引用相结合的数据组织方式, 箭头表示集合间引用；框内浅色块表示嵌入数组

量答卷。若将这些数据全部嵌入同一文档，会使文档不断膨胀，也不便于独立查询与修改。因此，系统通过 `ownerId`、`surveyId` 和 `respondentId` 等字段建立关联，并使用 `surveyVersion` 标识答卷所属的内容版本。

这种划分使问卷基本信息的修改、内容版本的发布和答卷的持续保存能够相对独立地进行。为控制嵌入文档的规模，系统还限制每个问卷版本最多包含 100 道题，每道题最多包含 50 个选项及 20 条跳转规则。

3.3 MongoDB 的适用性分析

在线问卷的数据具有明显的层次结构。一份问卷包含多道题目，每道题目又包含选项、校验条件和跳转规则。MongoDB 支持嵌套对象与数组，能够直接表示这种结构，使数据的存储方式与业务中的组织形式相对应。对于通常需要整体读取的问卷配置，这种方式可以减少额外的数据关联与组装工作。

不同题型也具有不同的配置需求。例如，多选题需要限制选择数量，数字填空题则需要设置取值范围及整数约束。MongoDB 的文档结构允许不同题型保存各自需要的配置字段，因此可以使用统一的题目结构组织不同类型的数据，并通过相应的校验逻辑保证配置的合法性。这种方式也为后续增加题型和规则提供了扩展空间。

同时，数据的组织方式还需要考虑其增长特点。题目、选项和规则的数量相对有限，适合嵌入问卷版本文档；历史版本和填写记录则可能持续增加，更适合独立存储并通过标识符建立关联。因此，本项目采用嵌入与引用相结合的方式，在完整数据读取、独立管理和后续扩展之间取得平衡。

3.4 索引、Schema 与数据一致性

在确定数据的组织方式后，系统还需要约束数据结构、优化常见查询，并保证关键操作在并发或中断情况下的正确性。

Schema 约束：系统使用 BSON 的 `ObjectId` 和 `Date` 类型保存标识与时间，通过 MongoDB 的 `$jsonSchema` 约束字段类型、必要属性及数组规模。例如，题目类型必须属于系统支持的四种类型，每个问卷版本最多包含 100 道题目。对于选择数量、数值范围以及跳转目标是否合法等业务规则，则由后端校验器进一步检查。通过结合数据库约束与业务校验，保证存储数据的结构与业务要求相符。

索引设计：系统根据实际查询方式，在四个集合上建立了相应索引，具体如表 3 所示。其中，用户名、问卷公开标识和问卷版本采用唯一索引，既支持快速定位，也避免相关标识重复；答卷集合则根据问卷、版本、填写者和提交时间建立复合索引，以支持历史查询与统计筛选。此外，通过提交请求标识的复合唯一索引识别重复请求，避免同一次提交被重复保存。考虑到索引会

增加存储和写入开销，系统主要针对常见查询及必要的唯一性约束建立索引。

并发控制：同一创建者可能在不同页面修改同一份草稿。为避免较早读取的内容覆盖后续修改，系统为草稿设置修订号 `revision`。每次保存时检查预期修订号和草稿状态，仅在条件匹配时更新内容并递增修订号；若草稿已被修改或冻结，则拒绝当前操作。

事务与提交一致性：创建问卷、派生草稿和提交答卷等跨文档操作使用 MongoDB 多文档事务。例如，提交答卷时，系统在事务中检查问卷状态、发布版本及答案合法性，同时保存填写记录并更新问卷的提交标识 `hasSubmissions`。首次成功提交后，系统禁止修改重复填写策略，并通过对共享问卷文档的更新协调提交与关闭、发布等并发操作。事务依赖 MongoDB Replica Set，本项目采用单节点副本集环境支持相关操作。

发布与历史一致性：系统通过版本冻结与发布指针管理问卷的历史内容，其关系如图 5 所示。发布时，首先完成题目及跳转规则的校验，将草稿版本冻结为 `sealed`，随后更新问卷的 `publishedVersion` 指针。若中途发生中断，已冻结的版本仍可保留，并通过重试完成发布，避免公开问卷指向尚未冻结的内容。

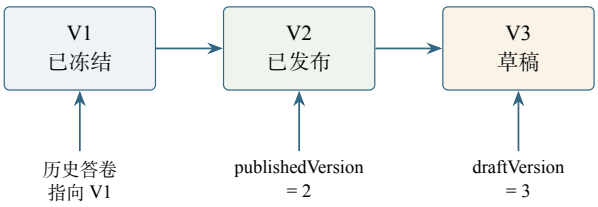


图 5. 版本指针与不可变历史快照

每份答卷同时记录提交时的 `surveyVersion`，使历史答案能够按照原版本的题目和规则解释。例如，图 5 中关联 V1 的历史答卷不会因 V2 发布或 V3 编辑而改变其含义。若填写者使用旧页面提交，而问卷已经发布新版本，系统会拒绝该请求，要求重新获取问卷。版本快照保留题目、规则及匿名配置，问卷标题、说明等基本信息仍由 `surveys` 独立管理。

3.5 答卷保存、有效统计与匿名保护

为支持用户重复填写并保留完整历史，系统采用**追加式存储（Append-only）**。每次新的有效提交都会生成独立的 `submissions` 文档，而不覆盖之前的答卷。例如，用户先后提交三次问卷，系统会保留三份记录，而不是仅保存最后一次的答案。这种方式能够保留历史数据，并支持后续采用不同的统计规则。同时，系统通过 `requestId` 和提交内容摘要识别重复请求，避免网络重试导致同一次提交被重复保存。

在此基础上，系统将**数据保存与统计有效性分离**。问卷通过 `submissionPolicy` 设置重复填写策略：`latest_only` 表示每位用户仅最后一次提交参与当前有效统计，`multiple` 则允许所有提交参与统计。对于存在多个版本的问卷，系统先根据 `respondentId` 和提交时间，在整份问卷的所有版本中确定每位用户的最新答卷，再筛选指定版本。例如，用户填写 V1 后又填写 V2，那么 V1 的旧答卷仍然保留，但不再参与 V1 的当前有效统计。若需要查看指定版本的全部历史提交，则可以使用 `historical_raw` 统计口径。为避免填写过程中改变统计规则，系统在首次成功提交后锁定重复填写策略。

对于匿名填写，系统将**内部身份记录与创建者的信息访问分离**。填写者仍需登录，系统通过 `respondentId` 识别用户，以支持重复填写判断和有效答卷筛选；但创建者查看匿名答卷时，接口不会返回填写者的账号标识。系统还在提交时保存 `anonymousAtSubmission`，记录该答卷对应的匿名设置。例如，V1 采用匿名填写，即使后续 V2 调整为非匿名，V1 的历史答卷仍按原有设置隐藏身份，避免后续修改影响已有数据的隐私属性。

4 跳转设计

问卷的填写流程可能随用户的答案而变化。例如，当用户选择“没有编程经验”时，可以跳过后续关于已掌握编程语言的问题，直接进入下一部分。如果将这些跳转条件直接写入程序，每次调整问卷流程都需要修改相应代码，难以适应不同问卷的配置需求。

因此，系统采用**基于配置的声明式规则设计**，将答案约束、跳转条件和目标位置作为结构化数据保存在问卷版本中，由统一的规则引擎解释执行。问卷创建者可以通过修改题目配置调整填写路径，而不必为每份问卷单独编写判断逻辑。同时，规则随问卷版本保存，使后续修改不会影响已有版本的填写逻辑。

4.1 统一的题目数据结构

为统一管理不同题型，系统为每道题目定义相同的基本结构，包括 `id`、`type`、`title`、`required`、`options`、`validation` 和 `routingRules`。其中，`type` 标识题目类型，`validation` 保存答案约束，`routingRules` 保存条件跳转规则。

不同题型共享上述结构，但可以根据自身特点设置不同的校验参数，如表 4 所示。

表 4. 题目类型与答案校验规则

题型	答案格式	主要校验条件
single_choice	字符串	必答检查、选项 ID 有效
multiple_choice	字符串数组	选项合法、无重复、选择数量限制
text	字符串	必答检查、文本长度范围
number	数字	数值范围、整数要求

例如，多选题通过 `minSelections` 和 `maxSelections` 限制选择数量，数字题则通过 `min`、`max` 和 `integerOnly` 约束输入。校验器根据题目类型读取对应参数，在统一流程中完成答案检查。这种设计既减少了不同题型之间的重复处理，也便于后续扩展新的题目类型与校验规则。

题目按照 `questions[]` 数组的顺序排列，未触发跳转规则时默认进入下一道题；需要跳转时，则通过稳定的题目 ID 指定目标。这样可以避免题目位置变化导致引用标识失效，同时在调整题目顺序后重新检查跳转目标，确保规则仍然符合向后跳转的要求。

表 3. MongoDB 主要索引设计及用途

集合	索引字段	设计目的
users	`username`（唯一）	快速定位用户，并防止用户名重复。
surveys	`publicId`（唯一）	通过公开链接查找问卷，保证链接标识唯一。
surveys	`(ownerId, createdAt)`（时间降序）	查询用户创建的问卷，并按创建时间排序。
survey_versions	`(surveyId, version)`（唯一）	定位指定问卷版本，防止版本编号重复。
submissions	`(surveyId, respondentId, requestId)`（唯一）	识别重复提交请求，避免同一次请求重复保存。
submissions	`(surveyId, submittedAt, _id)`（时间及 ID 降序）	按问卷查询历史答卷，并支持稳定排序。
submissions	`(surveyId, surveyVersion, submittedAt)`（时间降序）	查询指定问卷版本的填写记录。
submissions	`(respondentId, submittedAt)`（时间降序）	支持按填写者及提交时间查询记录。

表 5. 条件跳转规则的执行过程

行号

```
1  NEXT(questions, i, answer):
2    q ← questions[i]
3    rules ← SORT(q.routingRules, priority)
4    for rule in rules:
5      if MATCH(rule.condition, answer):
6        if rule.target.kind == end:
7          return END
8        return FIND(rule.target.questionId)
9    return i + 1
```

4.2 基于条件的跳转规则

系统将跳转规则保存在题目的 `routingRules[]` 中，每条规则主要包含三个部分：`condition` 描述触发条件，`target` 指定跳转目标，`priority` 决定多条规则的检查顺序。

不同题型支持相应的条件运算。单选题可以判断答案是否等于指定选项；多选题可以判断答案是否包含任一指定选项（`contains_any`），或同时包含全部指定选项（`contains_all`）；数字题则支持等于、大于、小于及相应的边界比较。当前文本题暂不支持条件跳转。

例如，对于一道编程语言多选题，可以设置以下两条规则：

- 优先级 1：同时选择 Python 和 Java，跳转至 Q4；
- 优先级 2：选择 C++，结束问卷。

当用户同时选择 Python、Java 和 C++ 时，两条规则均满足条件。为保证结果确定，系统**按照优先级从小到大检查规则，并执行第一条满足条件的规则**。因此，上述情况会优先跳转至 Q4；若所有规则均不满足，则默认进入下一道题。

规则引擎的核心执行过程如表 5 所示。

其中，`MATCH` 根据条件运算符判断答案是否满足规则，`FIND` 根据题目 ID 定位跳转目标。规则引擎只负责解释和执行配置，因此不同问卷可以复用同一套处理逻辑，创建者也可以通过调整规则配置修改填写流程。

为控制执行过程的复杂度，第一阶段仅允许跳转至后续题目或直接结束问卷。系统在保存题目配置时检查跳转目标是否存在且位于当前题目之后，从而避免循环跳转，使填写流程能够在有限步骤内结束。

4.3 规则校验与填写路径验证

条件跳转不仅需要正确执行，还需要保证规则配置与最终提交的答案相符。因此，系统分别对问卷配置和填写结果进行校验，避免无效规则影响填写过程，或将不符合跳转逻辑的答案保存到数据库中。

规则配置校验：系统在保存非空草稿、预览及发布问卷时，检查题目和规则是否有效，包括题目及选项 ID 是否重复、答案约束是否合理、条件运算符是否适用于对应题型，以及跳转目标是否存在、规则优先级是否冲突等。同时，跳转目标必须位于当前题目之后，避免出现循环路径。例如，单选题的跳转条件不能引用不存在的选项，数字题的最小值也不能大于最大值。通过提前检查，可以在问卷正式发布前发现并处理配置错误。

提交路径验证：前端根据规则引导用户填写，但最终提交的答案仍由后端独立校验。系统首先检查提交的问卷版本是否与当前发布版本一致，再根据该版本的题目配置，从第一道题开始逐题校验答案，并按照规则重新计算填写路径。其主要过程如图 6 所示。

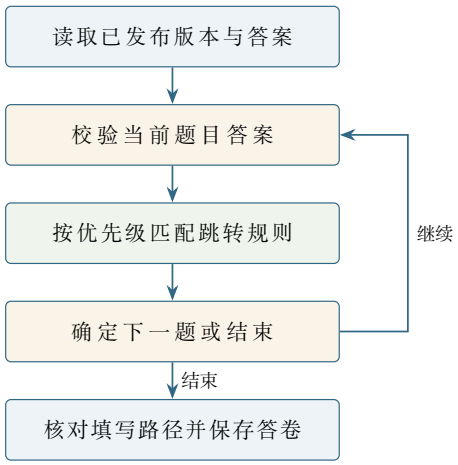


图 6. 提交时的答案与跳转路径验证流程

其中，必答约束只对实际经过的题目生效。例如，若用户在 Q1 选择“否”后直接跳转至 Q3，那么即使 Q2 被设置为必答题，也不需要回答 Q2。相反，如果用户提交了被跳过的 Q2 的答案，系统同样会拒绝该答卷。因此，我们规定**只有实际填写路径上的必答题才必须回答，且不能提交路径之外的答案**。

表 6. 核心测试场景与实际验证结果

测试类别	代表性用例与验证目的	实际结果
身份与权限	检查登录状态、越权访问、会话失效及 CSRF 防护，验证不同用户的数据访问边界。	未登录请求被拒绝；12 组问卷管理越权请求均返回 403；失效会话无法继续访问。
题目与跳转	测试四种题型的输入约束、规则优先级、向后跳转及路径外答案，验证规则执行的正确性。	合法路径正确重算；多条规则同时成立时按优先级执行；被跳过题目的答案被拒绝。
提交与幂等	使用相同请求标识并发提交 6 次，另由 4 名用户独立提交，验证重复请求不会产生重复答卷。	相同请求返回同一答卷 ID；最终保存 5 份有效提交，无重复写入。
版本与并发	测试草稿修订号冲突、发布中断恢复，以及提交与关闭、版本发布的并发情况。	4 个并发草稿修改仅 1 个成功；发布中断后可以恢复，旧发布版本保持可用。
匿名保护	测试匿名与实名版本切换、历史身份隔离及填写者用户名查询，验证身份展示规则。	V1 匿名答卷不返回账号标识；V2 实名答卷返回用户名，历史匿名属性保持不变。
结果统计	测试同一用户重复填写、多选频数及跨版本有效统计，验证统计分母与筛选口径。	原始提交 2 份、当前有效 1 份；跨版本提交后 V1 有效数为 0，历史原始数仍为 2。
浏览器流程	使用真实 Chromium 完成问卷编辑、预览、发布、填写、统计及手机端表单操作。	2 项端到端测试通过，完成桌面业务流程与 390px 手机页面验证。

通过配置阶段的规则检查与提交阶段的路径验证，系统能够在支持不同填写流程的同时，保证保存的答案与对应问卷版本的题目及跳转规则一致。

5 系统实现与验证

5.1 功能实现与结果统计

基于前述设计，系统已实现用户注册登录、问卷创建、可视化编辑、草稿预览、版本发布、公开填写及结果统计等主要功能。创建者可以配置四种题型、答案约束和条件跳转规则，填写者通过公开链接完成问卷，系统则根据已发布版本校验答案并保存提交记录。

在结果统计方面，系统使用 MongoDB 聚合管道完成选项频数、题目实际回答数及数字平均值的计算，具体实现见 统计服务源码。由于条件跳转可能使部分题目被跳过，各题的有效回答人数并不一定相同。因此，系统以实际回答该题的答卷数作为选项统计的分母，而不是统一采用问卷提交总数。对于多选题，一份答卷可以选择多个选项，因此各选项频数之和可能大于实际回答人数。

同时，系统区分原始提交、当前有效提交与报表所选记录。对于采用 `latest_only` 策略的问卷，先在所有版本中选出每位填写者的最新答卷，再筛选指定版本；需要查看完整历史时，则使用 `historical_raw` 口径。这样既保留了历史数据，也避免旧答卷重复参与当前统计，相关实现见 答卷处理源码。

例如，在实际集成测试中，同一用户连续提交两份答卷后，系统得到原始提交数为 2、当前有效提交数为 1。其中，最新答卷的多选题实际回答数为 1，而两个被选选项的频数之和为 2。随后用户填写新发布的 V2，V1 的当前有效提交数变为 0，但其历史原始提交数仍为 2。这些结果验证了题目统计分母、重复填写策略及跨版本筛选逻辑，相关断言见 集成测试源码，实际页面可查看 统计页面截图。

5.2 测试设计与结果

为验证系统是否满足课程要求，我们采用单元测试、MongoDB 集成测试和真实浏览器测试相结合的方式。单元测试重点检查题型约束与规则执行的正确性；集成测试验证数据库约束、接口行为、事务及并发一致性；浏览器测试则覆盖创建、发布、填写和统计的完整操作流程。对于匿名保护、版本切换和重复提交等容易产生数据错误的场景，进一步设计了边界用例。

主要测试内容及实际结果如表 6 所示。

根据仓库保存的最终执行记录，测试环境为 Windows、Python 3.11.5 和 MongoDB 8.3.11 单节点副本集。系统通过 `pytest` 执行全部测试，共包含 72 项单元测试、39 项数据库及接口集成测试，以及 2 项真实浏览器测试，最终结果为 **113 passed**、**0 failed**、**0 skipped**，总耗时 37.82 秒。

所有测试均有实际执行记录，相关证据可通过以下链接查看：测试源代码、测试报告、最终执行输出 及 JUnit 逐项结果。此外，系统还完成了 22 项真实 HTTP 请求检查，相关记录见 HTTP 验证结果；该检查独立于上述 113 项测试。

综上，测试结果表明，系统在当前实验环境下能够完成课程要求的主要业务流程，并正确处理已覆盖的跳转、版本、匿名及并发场景。对于 Docker 容器运行、大规模数据性能、多节点故障转移及公网部署等情况，目前尚未开展实际验证，因此不将其纳入本阶段的测试结论。

6 AI 使用与设计演进

本项目采用 AI 辅助开发，但数据库模型、业务规则和验收标准并非完全由 AI 自行决定。项目初期，我们结合课程要求，与 AI 讨论了问卷系统的数据组织方式、实体关系、版本管理及重复填写策略，并逐步形成明确的设计约束。相关规划过程可查看 前期设计讨论记录。

AI 的主要工作：在设计方案确定后，我们将需求整理为分阶段 Prompt，由 Coding Agent 辅助完成 Mon-

goDB Schema、规则引擎、后端接口、前端页面及自动化测试，并生成相应的技术文档。开发按照数据库与规则引擎、业务接口与网页、测试与验收三个阶段推进。实际使用的 Prompt 保存在 GitHub Prompt 目录，具体执行过程见 AI 使用日志。

自主设计与方案选择：我们的主要工作集中在需求分析和数据库架构决策上。在数据模型设计中，我们比较了将历史版本嵌入问卷文档、将版本独立存储，以及进一步拆分题目和选项等方案，最终选择由 `users`、`surveys`、`survey_versions` 和 `submissions` 组成的四集合模型。题目、选项与规则嵌入版本文档，问卷、版本及答卷之间通过标识符关联，以兼顾整体读取和历史数据的独立管理。同时，我们明确了匿名填写并非免登录、重复填写保留历史但可仅统计最新提交，以及历史版本冻结后不可直接修改等业务约束。这些决策构成了后续 AI 实现的基础。

AI 方案的不足与设计修正：在版本管理的方案讨论中，我们重点检查了历史版本如何存储、发布版本如何定位，以及不同版本的答卷如何统计。如果将持续增加的版本全部保存在同一问卷文档中，会增加文档增长与版本管理的负担。因此，我们采用独立版本文档，并通过 `publishedVersion` 和 `draftVersion` 管理当前状态。对于跨版本重复填写，我们进一步明确：`latest_only` 必须先在整份问卷的所有版本中选出每位用户的最新提交，再筛选指定版本，否则旧版答卷可能被错误计入当前有效统计。这些问题反映出，AI 提出的候选方案仍需要结合业务语义和数据增长特点进行审查，而不能只根据实现便利性作出选择。

实际问题与修改：开发和测试过程中也发现了具体问题。例如，数值校验在处理极大整数时存在潜在溢出风险，答卷明细曾直接展示选项内部 ID，初期 MongoDB 后台启动方式也未能在实际环境中持续运行。Coding Agent 根据审查及运行结果调整数值判断、选项展示和环境配置，并通过测试重新验证。此外，我们在检查非匿名答卷的展示方式后，进一步提出应显示注册用户名，而不是只显示内部用户 ID，同时要求保留历史匿名保护。随后由 Agent 完成最小范围修改，补充相关测试，最终全套测试达到 113 项通过。对应记录见 功能修正与测试证据。

通过本项目，我们认识到 AI 可以提高代码实现、测试构造和文档整理的效率，但不能替代对业务需求、数据一致性和设计取舍的判断。我们的工作重点是明确需求边界、比较并选择数据模型、检查 AI 方案中的潜在问题，并提出必要的修正要求；AI 则承担具体实现、测试执行和部分问题修复。最终通过设计文档、开发日志与实际测试记录，将这一过程保留为可追溯的项目证据。

7 遇到的问题与解决方法

在项目开发过程中，我们不仅遇到了代码实现问题，也发现部分业务需求需要进一步明确其数据语义。针对这些问题，我们结合数据库设计、代码审查与实际测试逐步完善方案。

历史版本与数据一致性：问卷发布后仍可能修改题目，但直接覆盖原有配置会影响历史答卷的解释。经过方案比较，我们决定将历史版本独立存储，采用“冻结版本—更新发布指针”的发布方式，并通过修订号和 MongoDB 事务协调并发操作。测试中进一步模拟发布中断，验证了旧版本的可用性及后续恢复能力。

重复填写与跨版本统计：最初需要解决的是用户多次填写时如何确定有效答卷。进一步分析发现，如果先筛选问卷版本再去重，可能使同一用户的旧版提交仍被视为有效。因此，我们采用追加式存储保留历史，并规定先跨版本选出每位用户的最新提交，再筛选目标版本。通过集成测试验证了历史记录保留与有效统计结果的一致性。

条件跳转与答案校验：引入跳转规则后，用户不一定经过全部题目，传统的逐题必答检查可能错误地拒绝合法答卷。为此，我们采用声明式规则，并由后端根据用户答案重新计算填写路径，要求只有路径上的必答题必须回答，同时拒绝提交被跳过题目的答案。这使答案校验能够与实际填写流程保持一致。

AI 实现中的边界问题：代码审查发现，极大整数可能在数值判断时引发溢出，答卷页面也曾直接展示内部选项 ID，影响可读性。随后调整数值校验顺序，并根据对应问卷版本映射选项标签。进一步检查实名答卷时，我们提出以用户名替代内部用户 ID 的展示要求，由 Agent 完成批量查询与匿名字段保护，并补充测试验证。相关修改过程见 AI 使用日志。

这些问题使我们认识到，数据库设计不仅需要考虑数据如何存储，还需要明确历史数据如何解释、业务规则如何保持一致，以及异常情况下如何保证正确性。AI 可以辅助实现与测试，但关键约束和方案取舍仍需要结合实际需求进行分析和判断。

8 结论与第二阶段展望

本项目完成了基于 MongoDB 的在线问卷系统，实现了用户管理、问卷编辑、版本发布、条件跳转、重复填写及结果统计等核心功能。在数据库设计中，我们采用四集合模型，将题目、选项和规则嵌入问卷版本文档，并将持续增长的答卷独立存储，在数据读取、历史管理和扩展性之间取得平衡。

本项目的核心设计在于**将问卷配置、历史版本与填写记录分离管理**。通过冻结已发布版本，保证历史答卷能够按照原有题目与规则解释；通过声明式规则引擎支持灵活的填写流程；通过事务、幂等机制及跨版本统计规则，维护提交与统计结果的一致性。相关功能已通过实际测试验证，为后续开发提供了较为稳定的基础。

第二阶段可在现有架构上进一步增加题型、跳转条件和统计指标，主要扩展相应的数据模型、规则校验、前端交互和统计服务，而不必重新设计整个数据库。同时，应保持历史版本的数据结构与解释语义兼容，避免新功能影响已有答卷。对于未来可能出现的大规模数据与高并发需求，还需要进一步评估索引效率、文档增长及事务竞争等问题。

通过本次课程设计，我们不仅完成了系统实现，也进一步理解了数据库设计需要综合考虑业务需求、数据增长、一致性约束与未来扩展，而不能仅以当前功能是否能够运行为判断标准。